



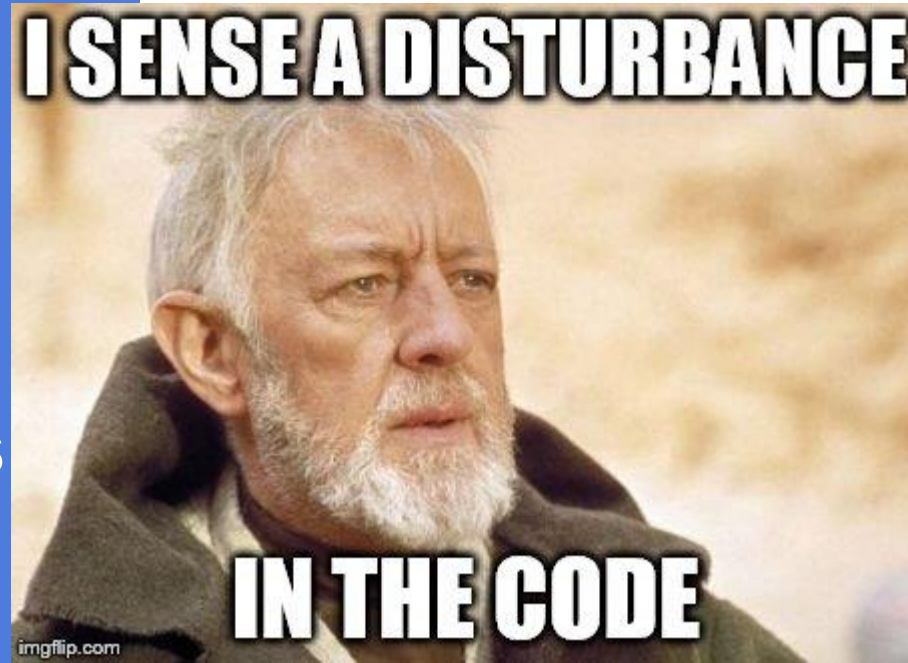
DuocUC[®] INFORMÁTICA Y
TELECOMUNICACIONES



- **Arquitectura de
Software Funcional**
-

Discutamos...

- ¿Por qué es importante que el código que programamos siga una “Estructura”?



THE EVOLUTION OF SOFTWARE ARCHITECTURE

1990's

SPAGHETTI-ORIENTED
ARCHITECTURE
(aka Copy & Paste)



2000's

LASAGNA-ORIENTED
ARCHITECTURE
(aka Layered Monolith)



2010's

RAVIOLI-ORIENTED
ARCHITECTURE
(aka Microservices)



WHAT'S NEXT?
PROBABLY PIZZA-ORIENTED ARCHITECTURE

By @benorama

A black and white photograph of a man in a suit standing in a modern office, holding a tablet and smiling. The office has glass partitions and modern furniture. A blue square is visible in the upper right area of the image.

01

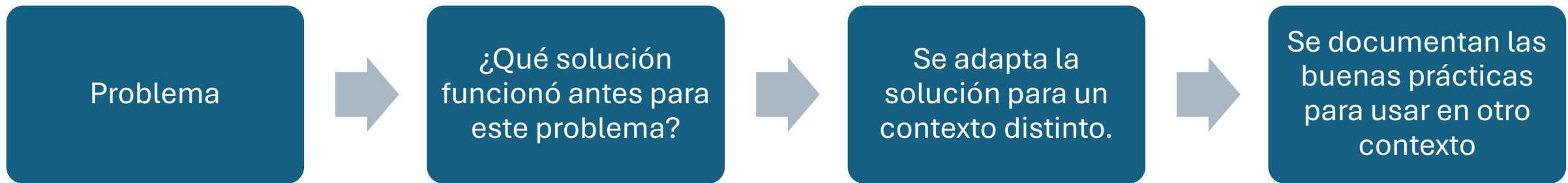
- **Arquitectura de Software y Patrones**

- **Qué es una “Arquitectura de Sistema”**
- **ISO/IEC/IEEE 42010 – Descripción de la arquitectura**
- Una arquitectura de sistema establece los conceptos o propiedades fundamentales de un sistema en su entorno incorporados en sus elementos, relaciones y en los principios de su diseño y evolución.

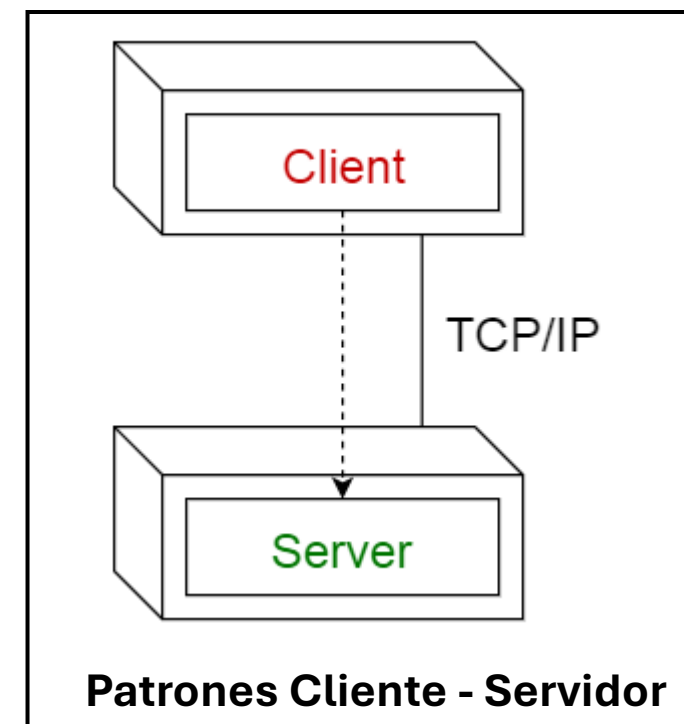
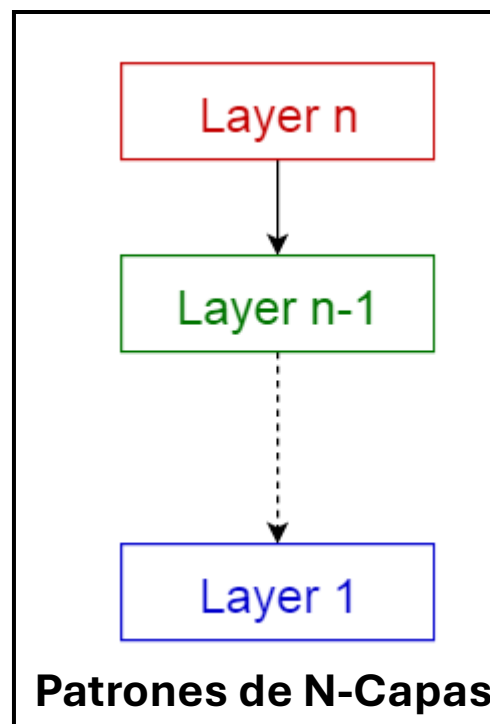
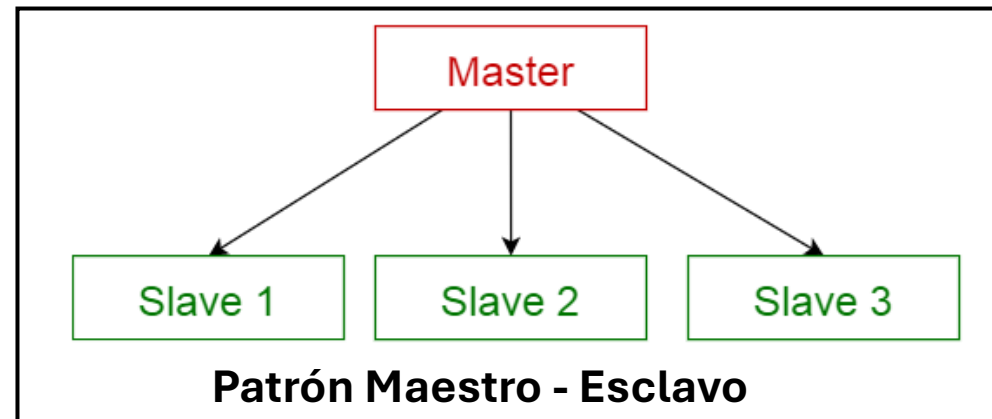
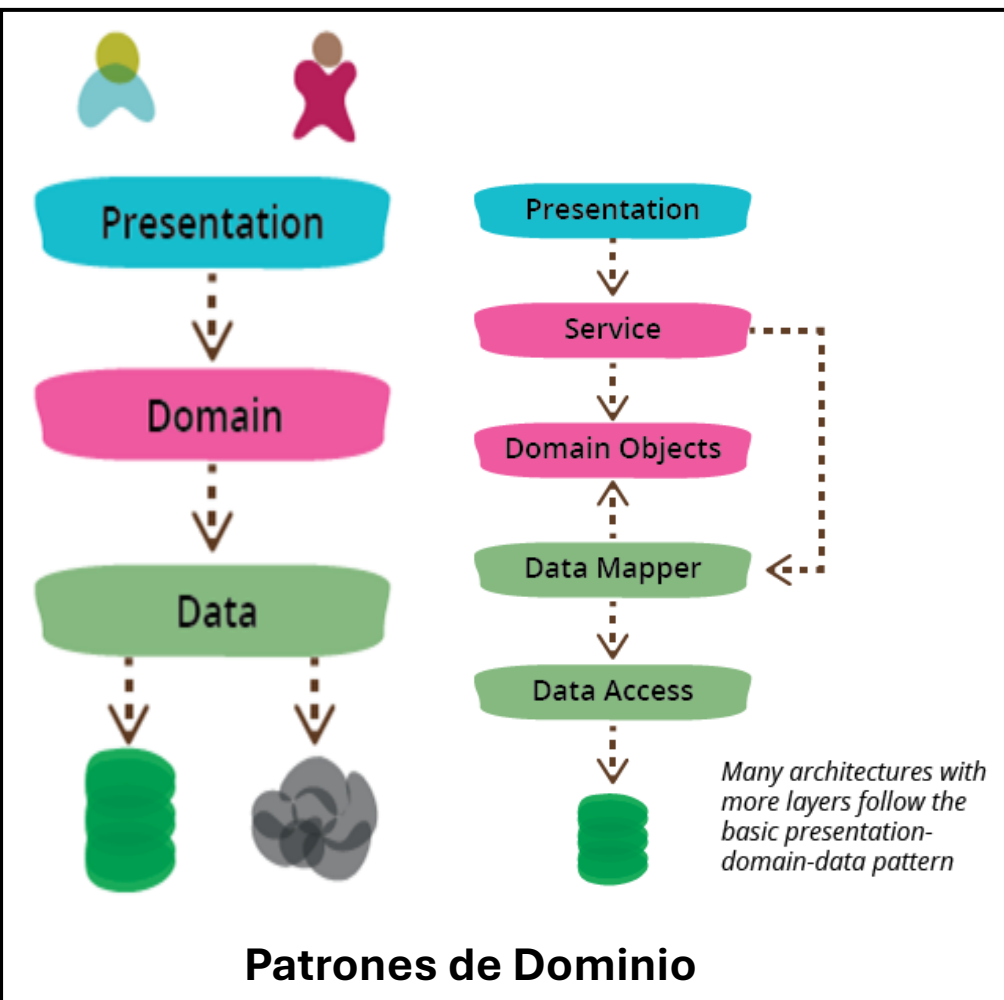
Una buena Arquitectura de Sistema presenta las siguientes características

- Satisface las necesidades de las partes interesadas y ofrece valor
- Se integra fácilmente con otras entidades (otros sistemas)
- Evoluciona de manera flexible para cumplir con los requisitos cambiantes a lo largo del tiempo
- Funciona de forma sencilla y fiable

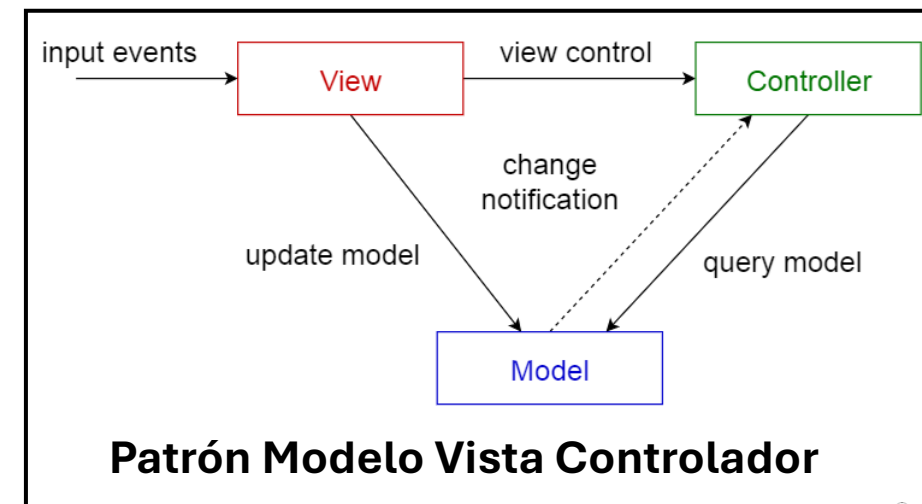
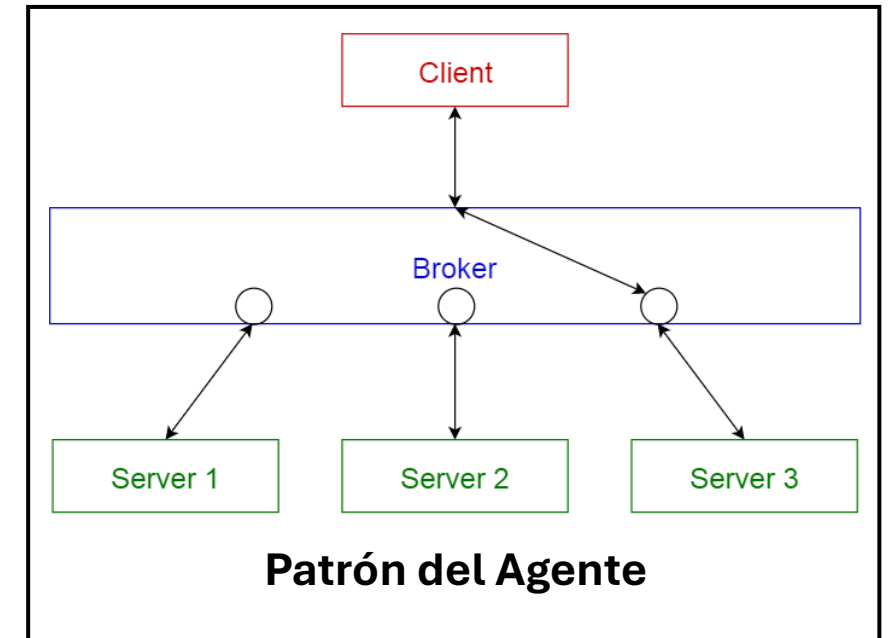
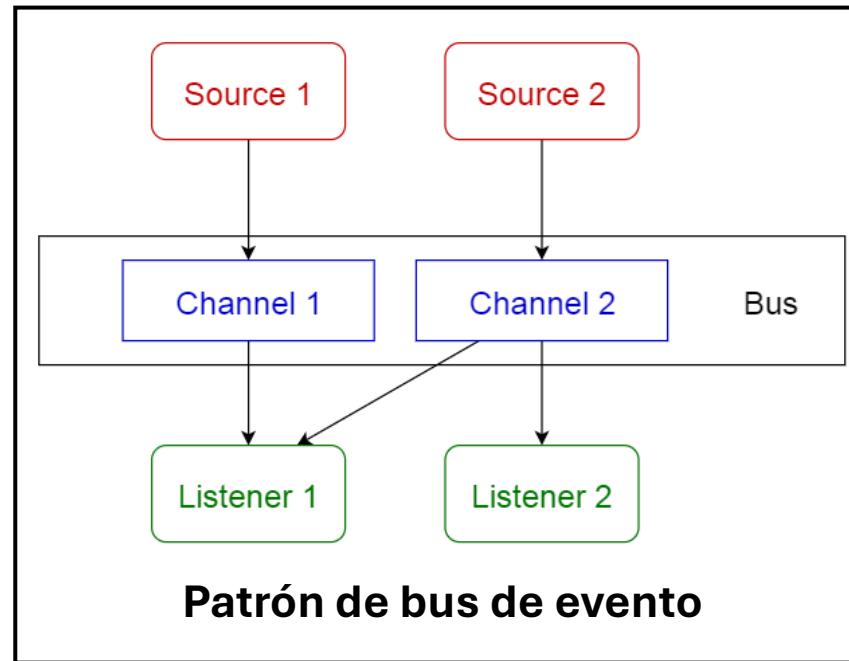
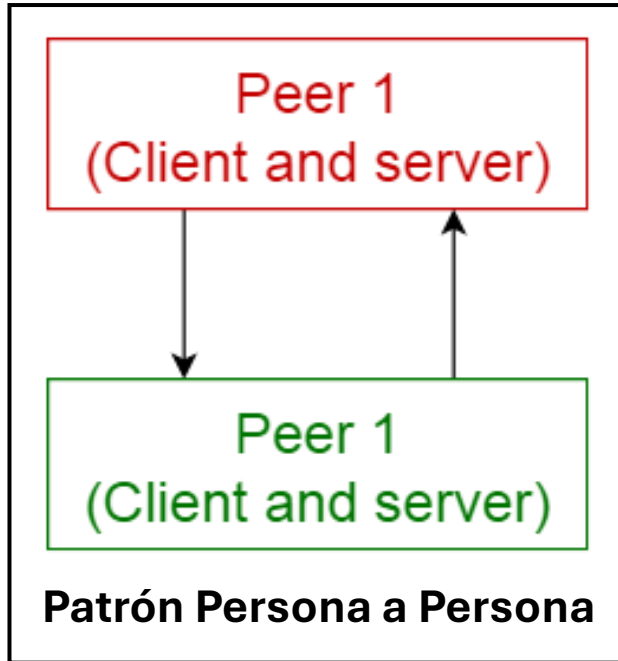
• ¿Qué es un Patrón?



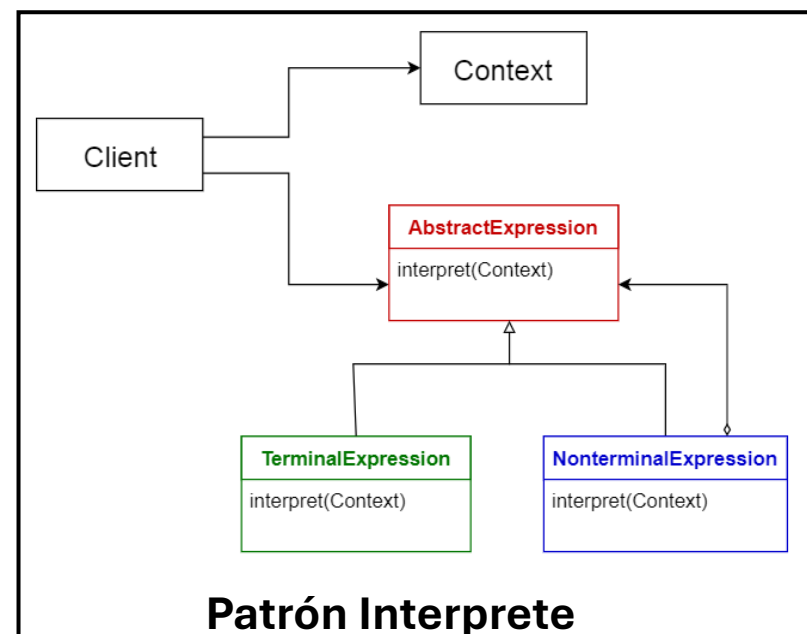
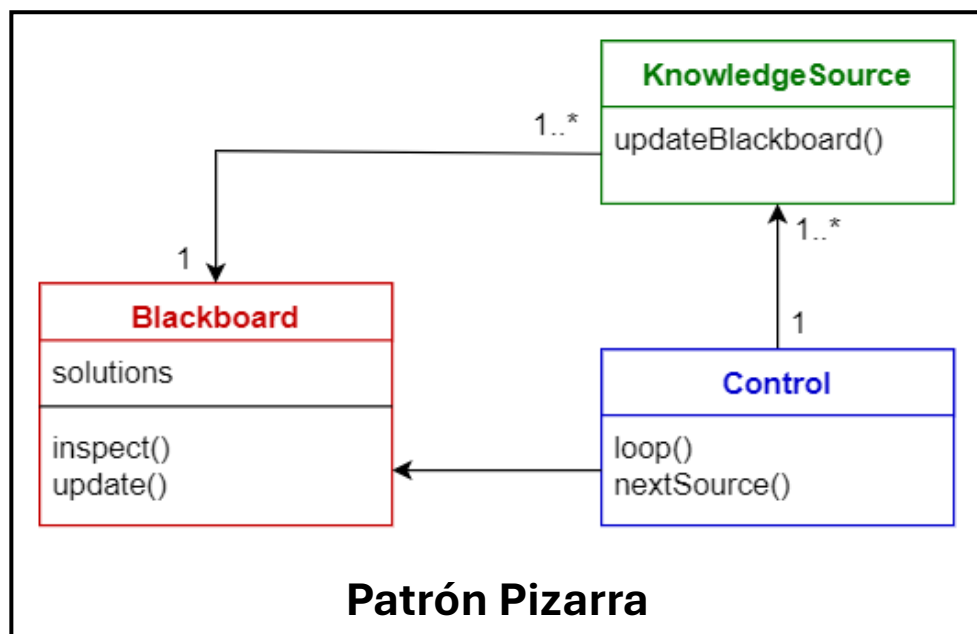
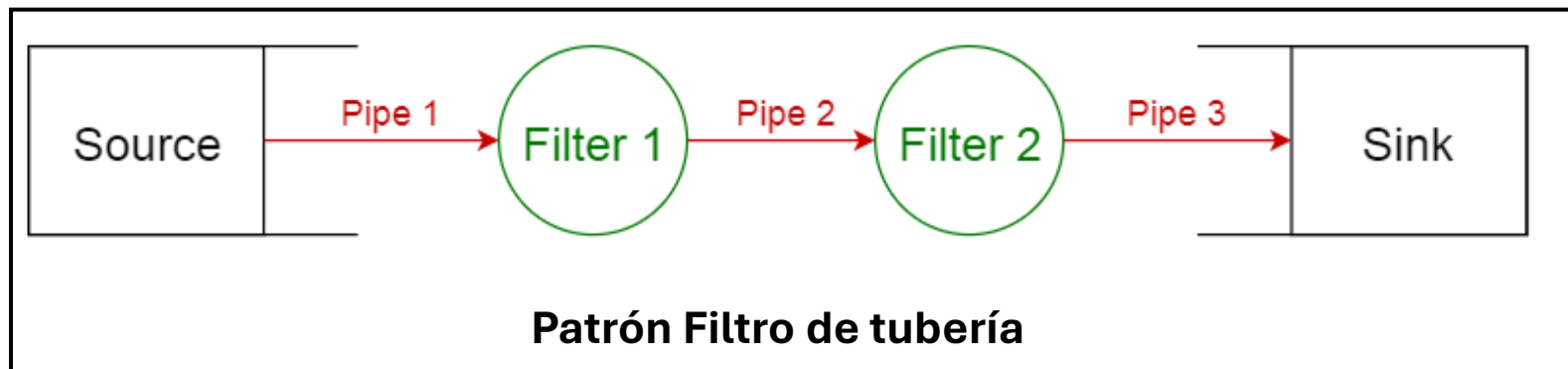
• Tipos de patrones



• Tipos de patrones



• Tipos de patrones



- **Micro Break**



**¡1 Cosa!
¿Qué recuerdas?**

¿Qué rescato o creo que es importante recordar sobre la arquitectura de software?



02

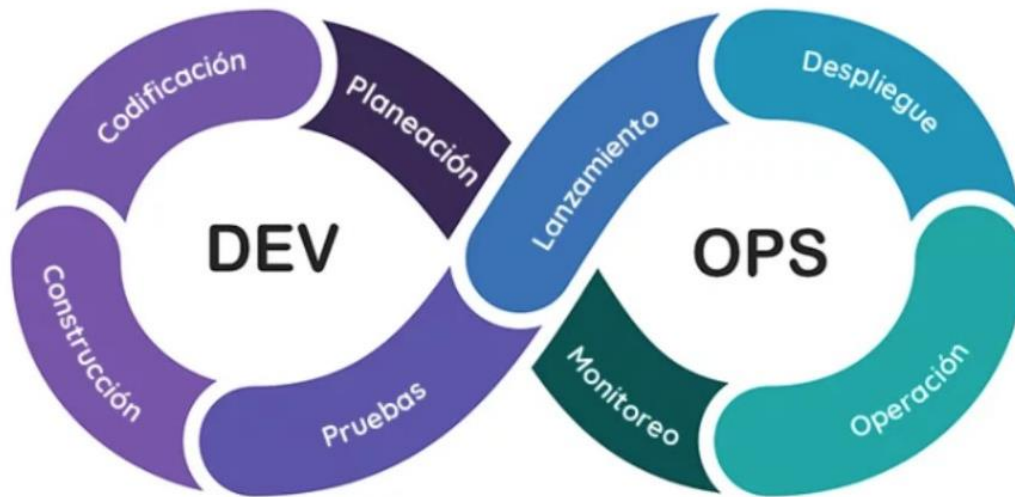
- **Arquitectura de Software Basada en Dominio**

• Tipos de Arquitectura

- Existen varios tipos de arquitectura, las cuales buscan resolver distintos problemas, y las cuales varían en complejidad, entre estos tipos no existe el concepto de mejor arquitectura, ya que todo “**depende**” **del contexto y del problema** que queremos resolver. Existen distintos tipos de arquitectura, donde podemos destacar a grandes rasgos los **monolitos** y las arquitecturas **distribuidas**.

• Arquitectura Ágil

- **La arquitectura no está escrita en piedra** y no es algo rígido, más bien es algo que va evolucionando a medida que el producto se desarrolla y no es trivial identificar dónde debería ir cierta lógica de negocios sin una arquitectura bien definida.



<https://sesitdigital.com/wp-content/uploads/2019/11/Imagen-destacada-SES-39.png>

- Los ingenieros de **DevOps** mejoran la colaboración entre las operaciones y los equipos de desarrollo. Utilizan tecnología, especialmente herramientas de automatización que pueden **aprovechar una infraestructura cada vez más programable y dinámica desde una perspectiva de ciclo de vida**

<https://sesitdigital.com/arquitectura-de-software-y-cultura-devops/>

• Tipos de Arquitectura

Monolítica

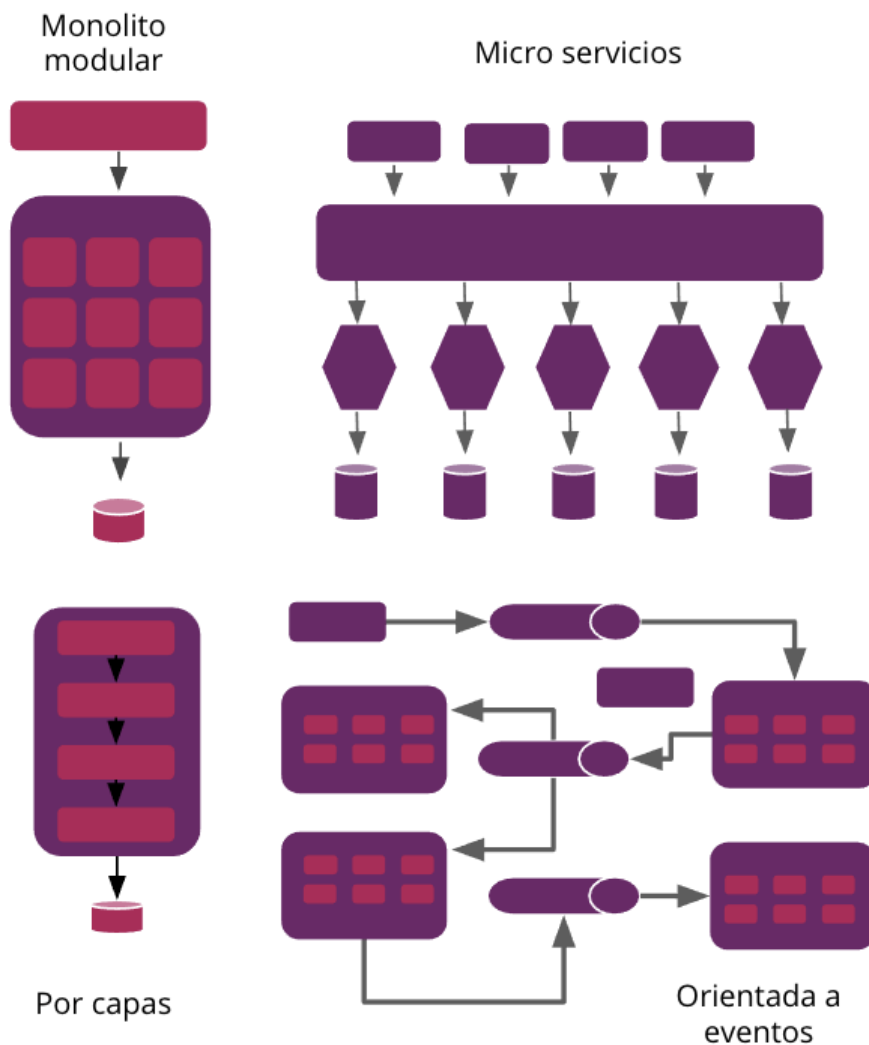
Los monolitos son aquellos que combinan la capa de interfaz del usuario, la capa de negocios y la base de datos en un solo sistema, es decir, es responsable por todos los pasos necesarios para una función en particular. En los monolitos podemos encontrar:

- Arquitectura por capas
- Arquitecturas modulares
- Micro Kernels

Distribuida

Las arquitecturas distribuidas son aquellas que al contrario del monolito sus procesos son distribuidos, es decir si realizamos una consulta en la base de datos, vamos a dividir todos los procesos en distintos nodos, lo que mejorará el performance final de la aplicación. En esta arquitectura podemos encontrar:

- Microservicios
- Arquitectura orientada a servicios
- Arquitectura orientada a eventos



• Principios para trazar límites de servicios

Elige **monolito** cuando:

- No existe una cultura de DevOps, es importante que las personas sean capaces de desarrollar esta habilidad antes de migrar del monolito
- Estás prototipando una idea y necesitas llevarla a producción de forma rápida
- Código existente presente en una sola unidad de despliegue

Elige microservicios cuando :

- Tu equipo es maduro en el negocio y la empresa tiene una cultura de DevOps
- Las personas desarrolladoras de tu equipo tienen experiencia en microservicios y entienden los conceptos de DDD (a veces adoptamos lo que está de moda sin leer los fundamentos, lo que puede llevar a equivocaciones de base que pueden impactar el negocio)
- Cuando necesitas unidades de despliegue independientes y escalar de forma horizontal

• Micro Break



**¡1 Cosa!
¿Qué recuerdas?**

¿Qué rescato o creo que es importante recordar sobre la Arquitectura basada en Dominio?

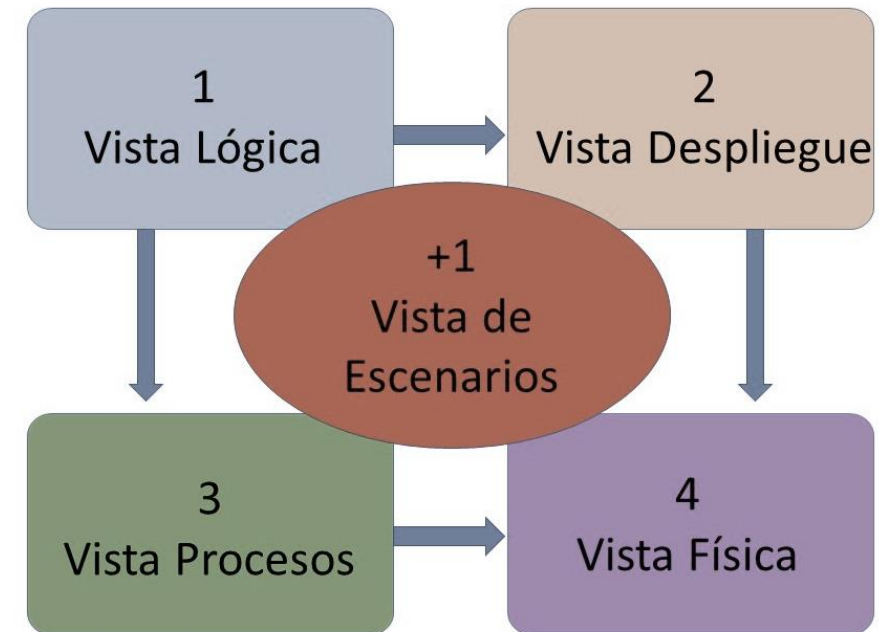


03

- Modelando la Arquitectura de Software

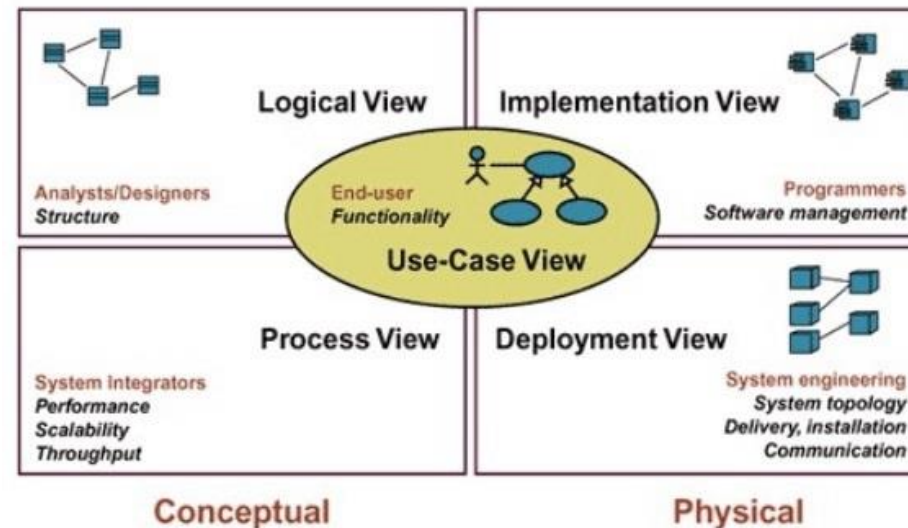
- **Modelo 4+1**

- El **modelo de vistas 4+1**, nos permitirá modelar nuestro software tomando como referencia un **enfoque centrado en la vista de los escenarios** como representación de las funcionalidades del software y a partir de esa definición, obtener visiones en diversos ámbitos del producto de software.



- **Modelo 4+1**

- El **modelo 4+1** es un modelo de vistas diseñado por el profesor **Philippe Kruchten** y se utiliza para **describir la arquitectura de un sistema** de software basado en el uso de múltiples puntos de vista, una vista nos indica **que es lo que se debe modelar** y un **punto de vista** son las **notaciones o normas para documentar** una o más vistas.



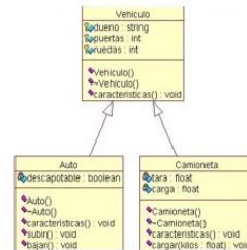
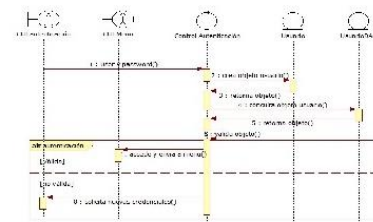
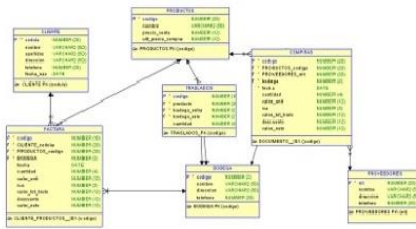
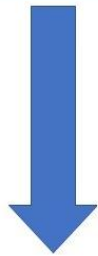
- **Vista Lógica**



- La **vista lógica** representa la estructura y función que el sistema proporcionará a los usuarios finales, es decir, se ha de representar **lo que el sistema realizará y las funciones y servicios que ofrece.**



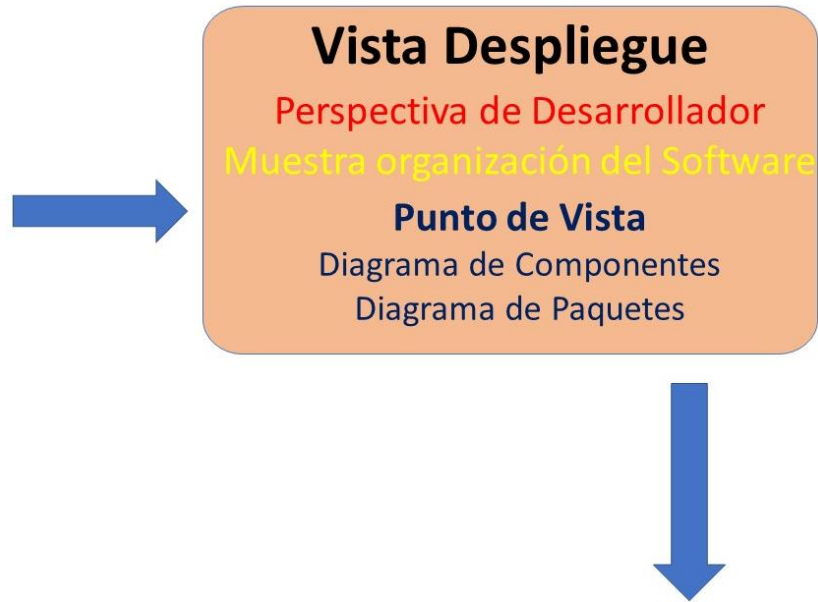
• Vista Lógica



Para completar la documentación de esta vista se pueden utilizar como punto de vista la notación que nos proveen los **diagramas de clases, de comunicación y de secuencia de UML**, también se pueden considerar como otro punto de vista la notación que proveen los **modelos de datos entidad-relación**.

La vista lógica está orientada a proveer información a los usuarios y todos quienes requieran conocer las funcionalidades del software.

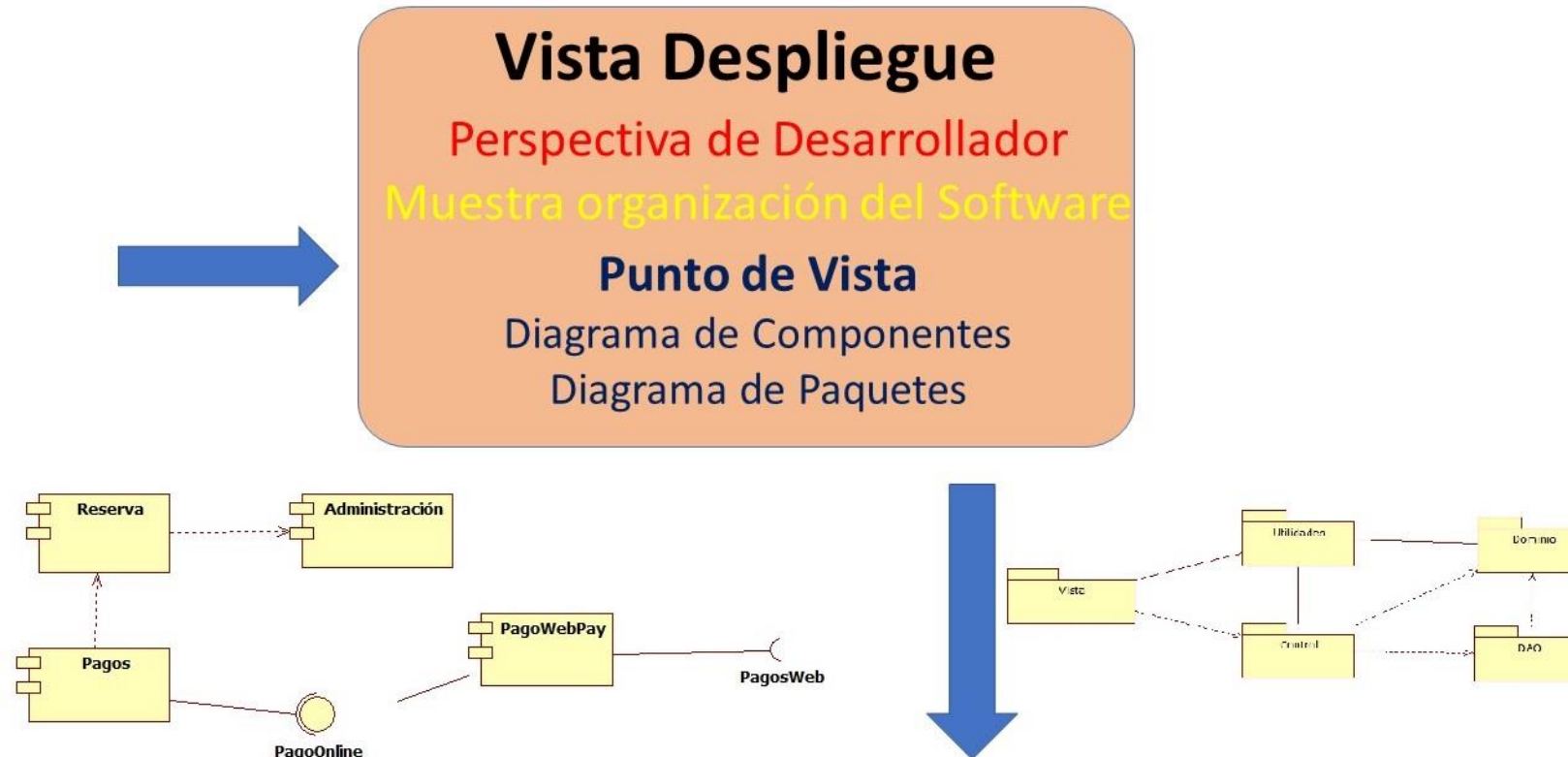
- **Vista Despliegue**



- La **vista de despliegue** muestra cómo está **dividido el sistema de software**, tanto sus **componentes como las dependencias** que hay entre estos componentes, además muestra como está **organizado el código fuente** y bibliotecas de funcionalidades adicionales o incluidas en la aplicación, mediante UML se puede utilizar **diagramas de paquetes y diagramas de componentes**.

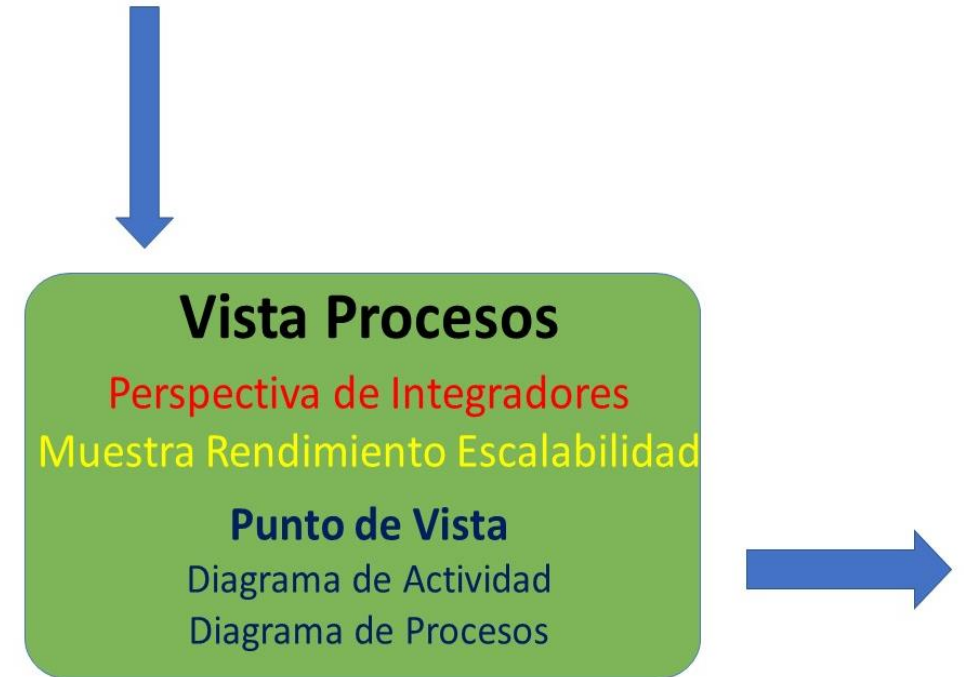
- **Vista Despliegue**

- La vista de despliegue muestra el software desde la **perspectiva de un desarrollador de software**.



- **Vista de procesos**

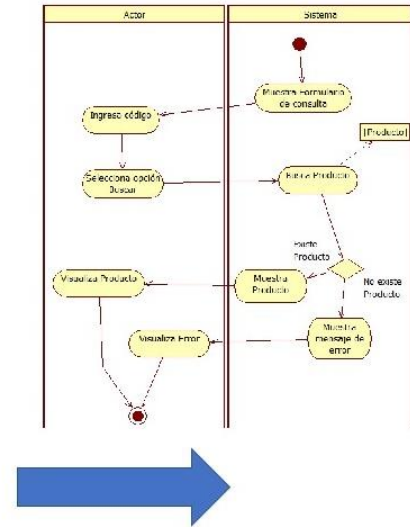
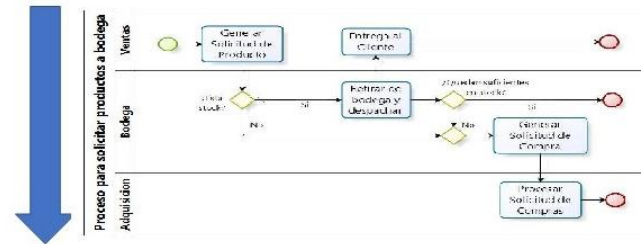
- La **vista de procesos** muestra los procesos que hay en el sistema y la **forma en la que se comunican estos procesos**, los flujos de trabajo de componentes negocio y operacionales que conforman el sistema logrando establecer un **rendimiento y escalabilidad**.



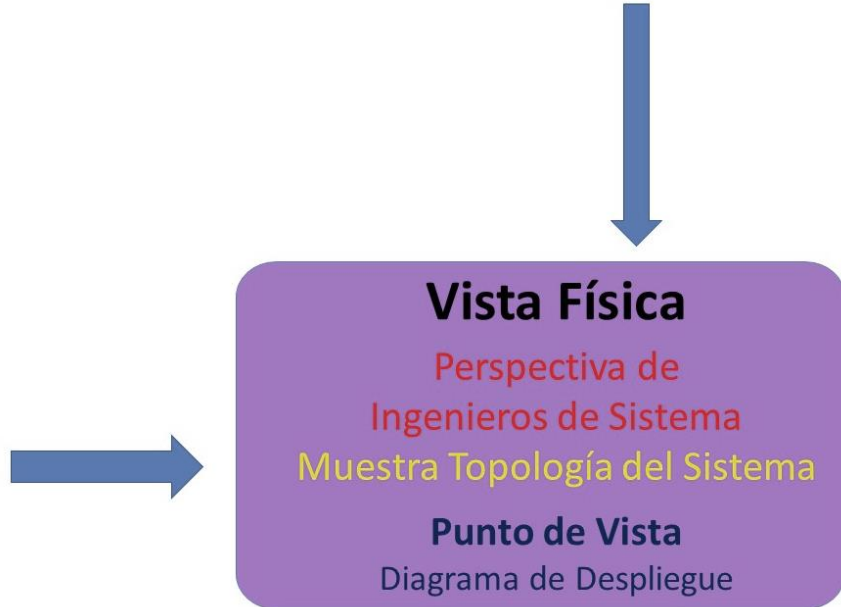
• Vista de procesos

Se pueden utilizar diversos puntos de vistas para documentar la vista de procesos, por ejemplo, utilizando **notación de procesos** a través de diagramas **BPMN** y mediante UML, el **diagrama de actividades** es el punto de vista que más se ajusta para su representación.

Orientada a los integradores de la aplicación, que tendrán de sincronizar los diversos procesos del sistema para lograr su operación.



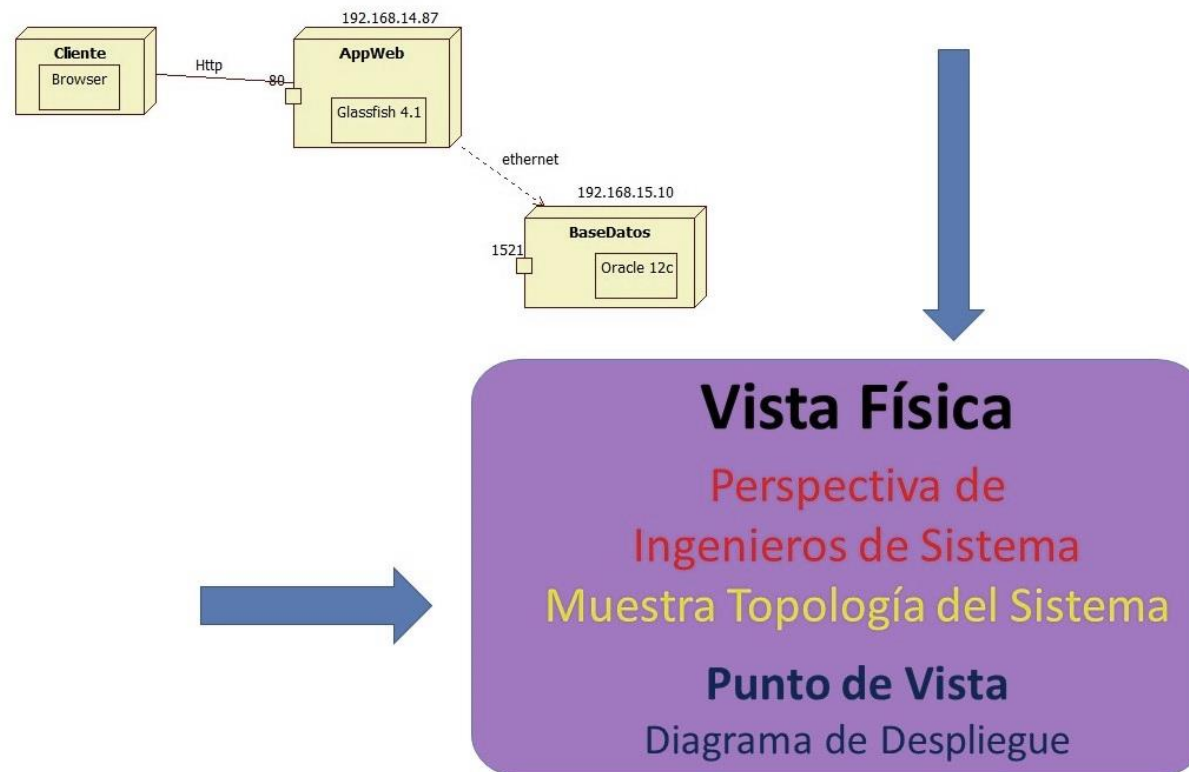
- **Vista física**



La **vista física** muestra los **componentes físicos del sistema** y las **conexiones entre dichos componentes**, los cuales conforman la solución de software, se incluyen además diversos **servicios** que la aplicación puede consumir.

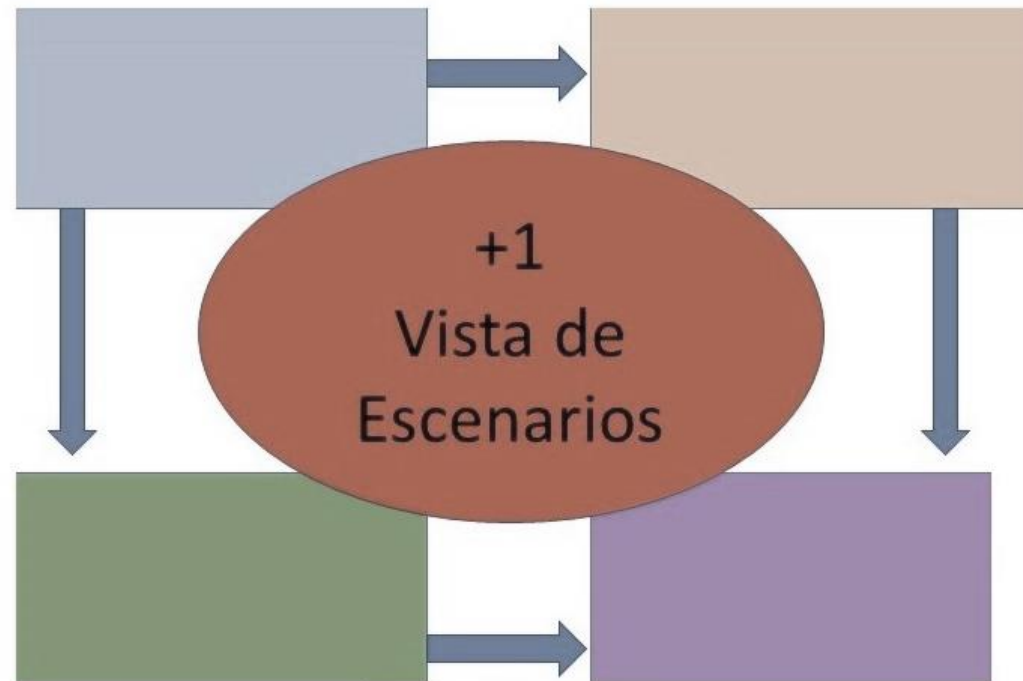
Orientada a los ingenieros de sistemas y de infraestructura, los cuales deben integrar elementos de hardware, software y orquestar servicios para lograr soportar la ejecución de la aplicación y el almacenamiento de sus respectivos datos.

- **Vista física**
- El punto de vista puede estar representado por el **diagrama de despliegue de UML**.

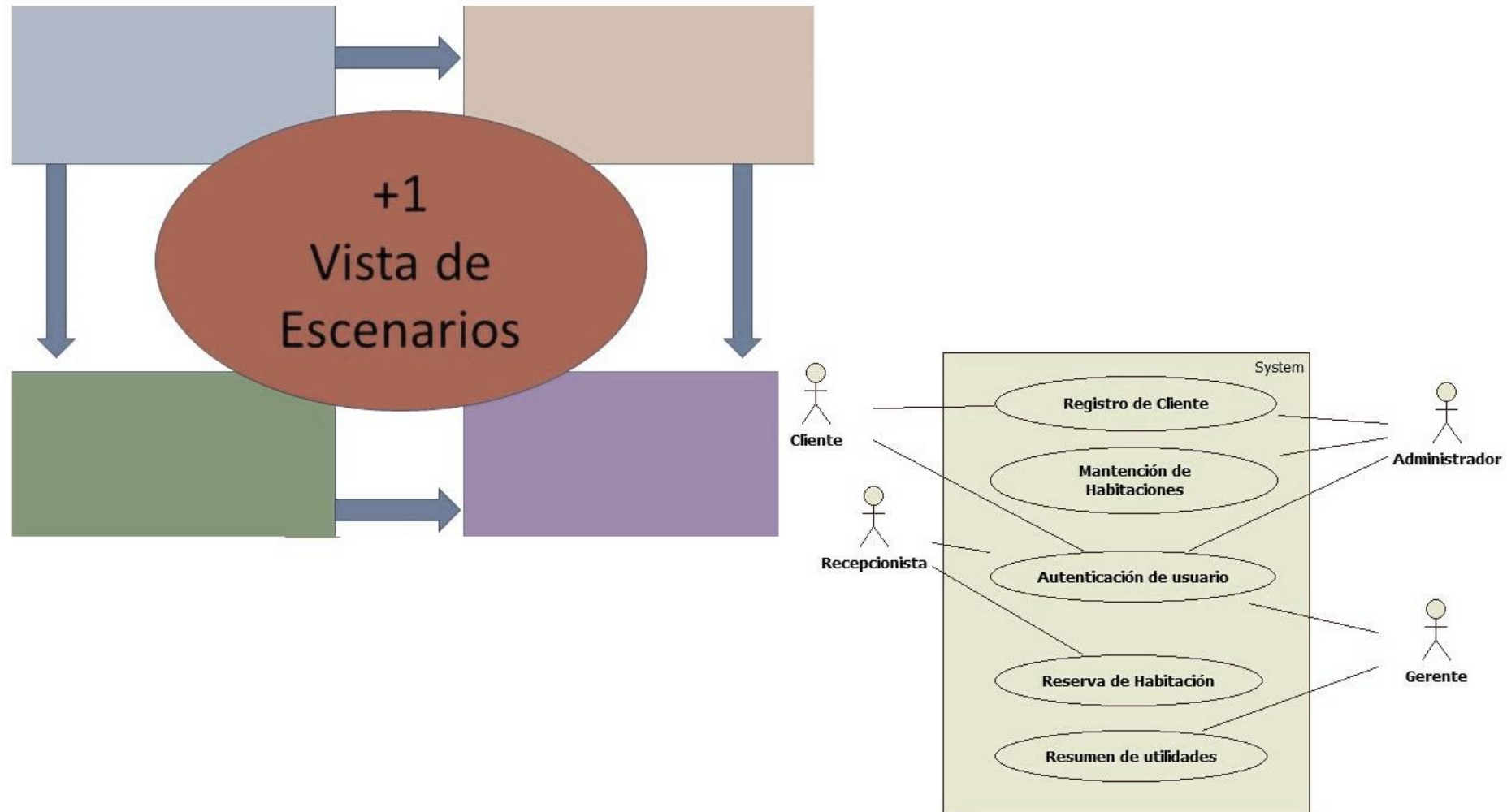


- **Vista de escenarios**

- La **última vista del modelo 4+1** se refiere a la **vista más uno**, que corresponde a la **vista de escenarios**, esta considera los aspectos relacionados a los **casos de uso** que serán cumplidos por el software y se **vincula con las otras 4 vistas**.



- **Vista de escenarios**



- **Micro Break**

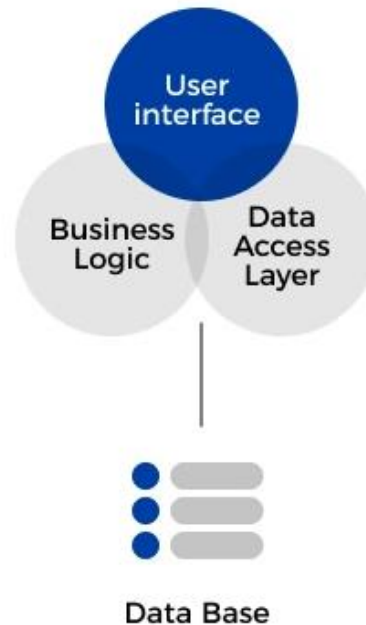


**¡1 Cosa!
¿Qué recuerdas?**

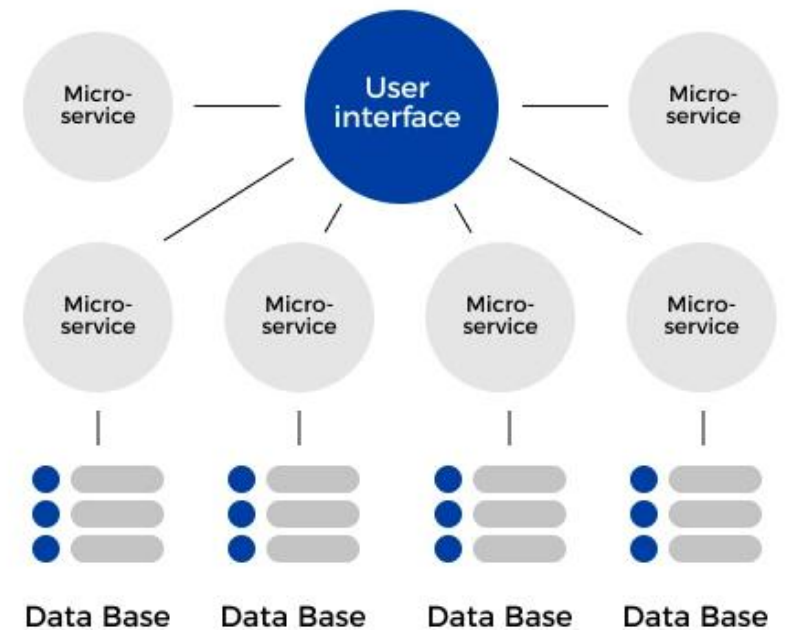
¿Qué rescato o creo que es importante recordar sobre el modelo 4+1?

Demuestra lo
que aprendiste

MONOLITHIC ARCHITECTURE



MICROSERVICE ARCHITECTURE



Arquitectura de Software y Patrones: ¿Cuál es la diferencia entre una arquitectura monolítica y una arquitectura de microservicios (**Ventajas y Desventajas**)? Proporcione un ejemplo de cada uno.

Demuestra lo que aprendiste



Modelado de la Arquitectura : Al momento de modelar, diseñar y construir la arquitectura de un software ¿es más importante aquello que es más visible para el usuario , porque eso es lo que mide la calidad de lo que hacemos?

04

- Conclusiones y Resumen

• Resumen

- **Arquitectura de Software y Patrones:** La arquitectura de software define la estructura de un sistema, sus componentes y sus relaciones. Los patrones de diseño son soluciones recurrentes a problemas comunes en el diseño de software, proporcionando una guía reutilizable y probada para crear software robusto y mantenible.
- **Arquitectura Basada en Dominios:** Esta arquitectura organiza el sistema en módulos basados en dominios específicos del negocio, promoviendo la separación de preocupaciones y facilitando la alineación del software con las necesidades del negocio. Utiliza conceptos como el modelo de dominio y el lenguaje ubicuo para mejorar la comunicación entre desarrolladores y expertos del dominio.
- **Modelado 4+1:** El modelo 4+1 describe la arquitectura de software utilizando cinco vistas diferentes: lógica, de desarrollo, de proceso, física y de escenarios. Estas vistas permiten a los diferentes interesados (desarrolladores, usuarios, operadores) entender y validar el diseño del sistema desde múltiples perspectivas, asegurando una cobertura integral de los requisitos y funcionalidades.

• Próxima clase

La próxima clase trabajaremos en una actividad práctica basada en la arquitectura de software



DuocUC[®]

CERCANÍA. LIDERAZGO. FUTURO.

duoc.cl

7 AÑOS
ACREDITADO



DESDE AGOSTO 2017 HASTA AGOSTO 2024.
DOCENCIA DE PREGRADO. GESTIÓN
INSTITUCIONAL. VINCULACIÓN CON EL MEDIO.